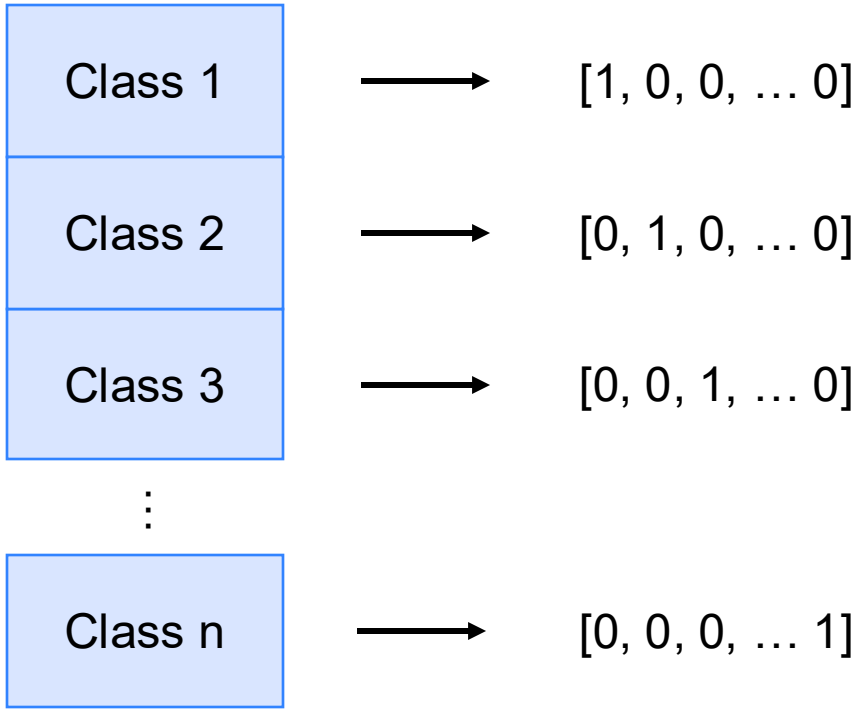
Artificial Intelligence in Interactive Systems

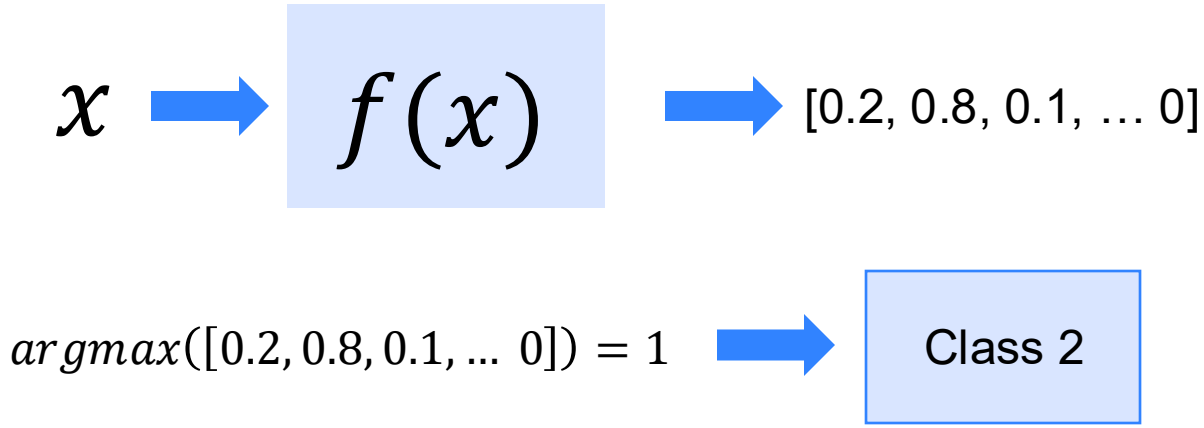
Neural Network Structure

Sven Mayer

One-Hot Encoding

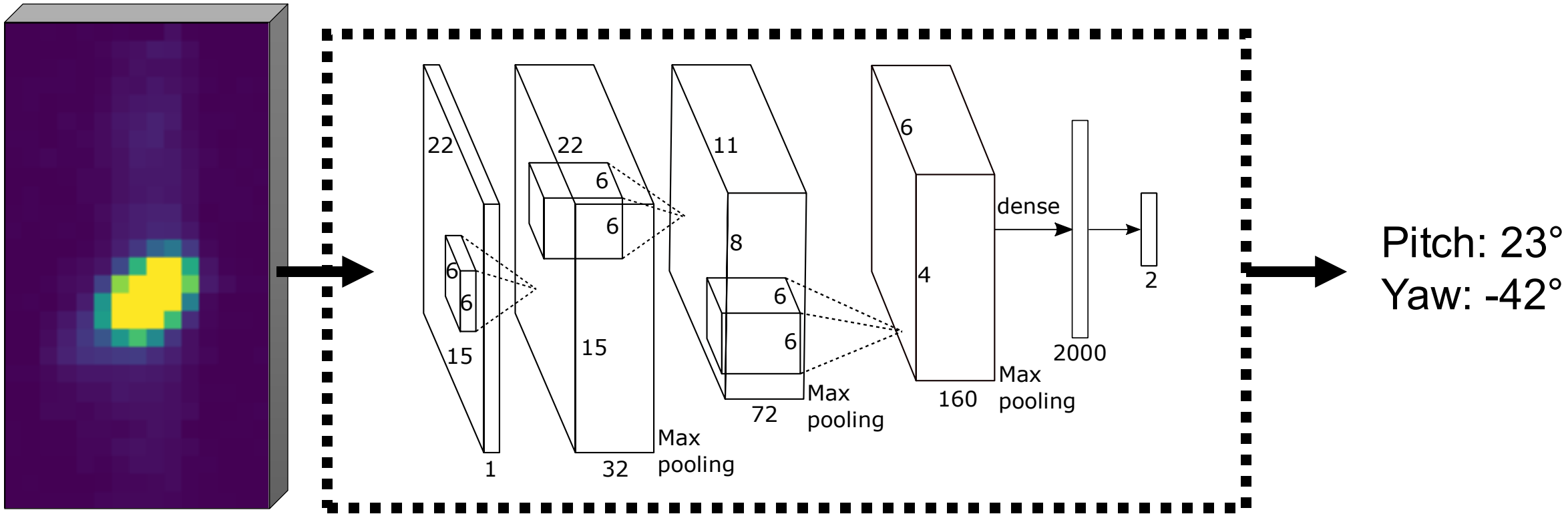
Classification





While the vectors are not, thinking about them like probabilities for one class helps to understand the concept.

Machine Learning to Enhance Sensing



S. Mayer, H. V. Le, & N. Henze (2017) Estimating the Finger Orientation on Capacitive Touchscreens Using Convolutional Neural Networks. In Proc. of ISS.
DOI: https://doi.org/10.1145/3132272.3134130

Neuronal Network Structure

TensorFlow Syntax for a 2-Layer Model

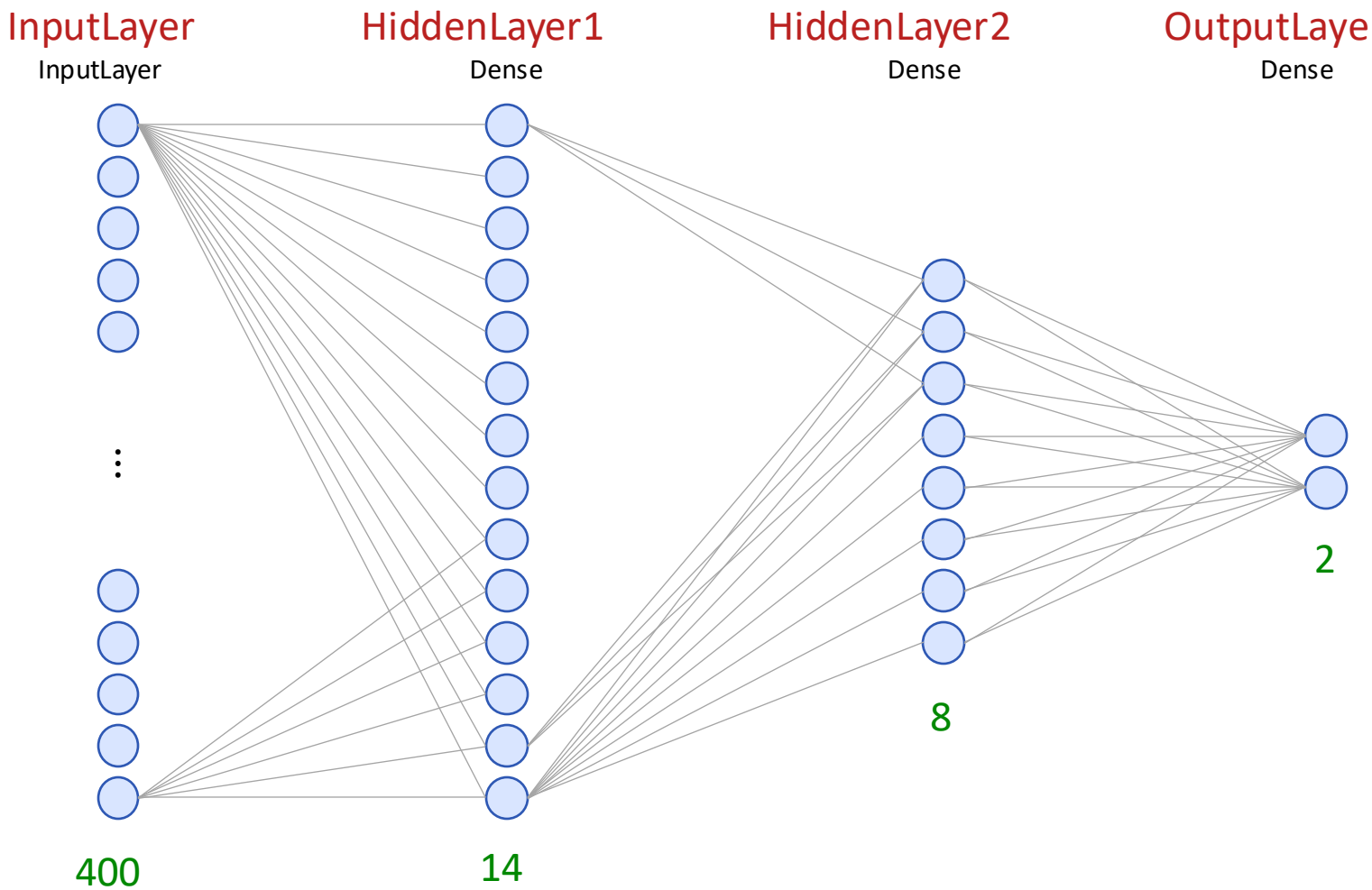
```
model = tf.keras.Sequential()

x = tf.keras.layers.InputLayer((400,), name = "InputLayer")
model.add(x)

x = tf.keras.layers.Dense(14, name = "HiddenLayer1", activation = 'relu')
model.add(x)

model.add(tf.keras.layers.Dense(8, name = "HiddenLayer2", activation='relu'))

model.add(tf.keras.layers.Dense(2, name = "OutputLayer", activation = 'softmax'))
```

Layers to Pick from

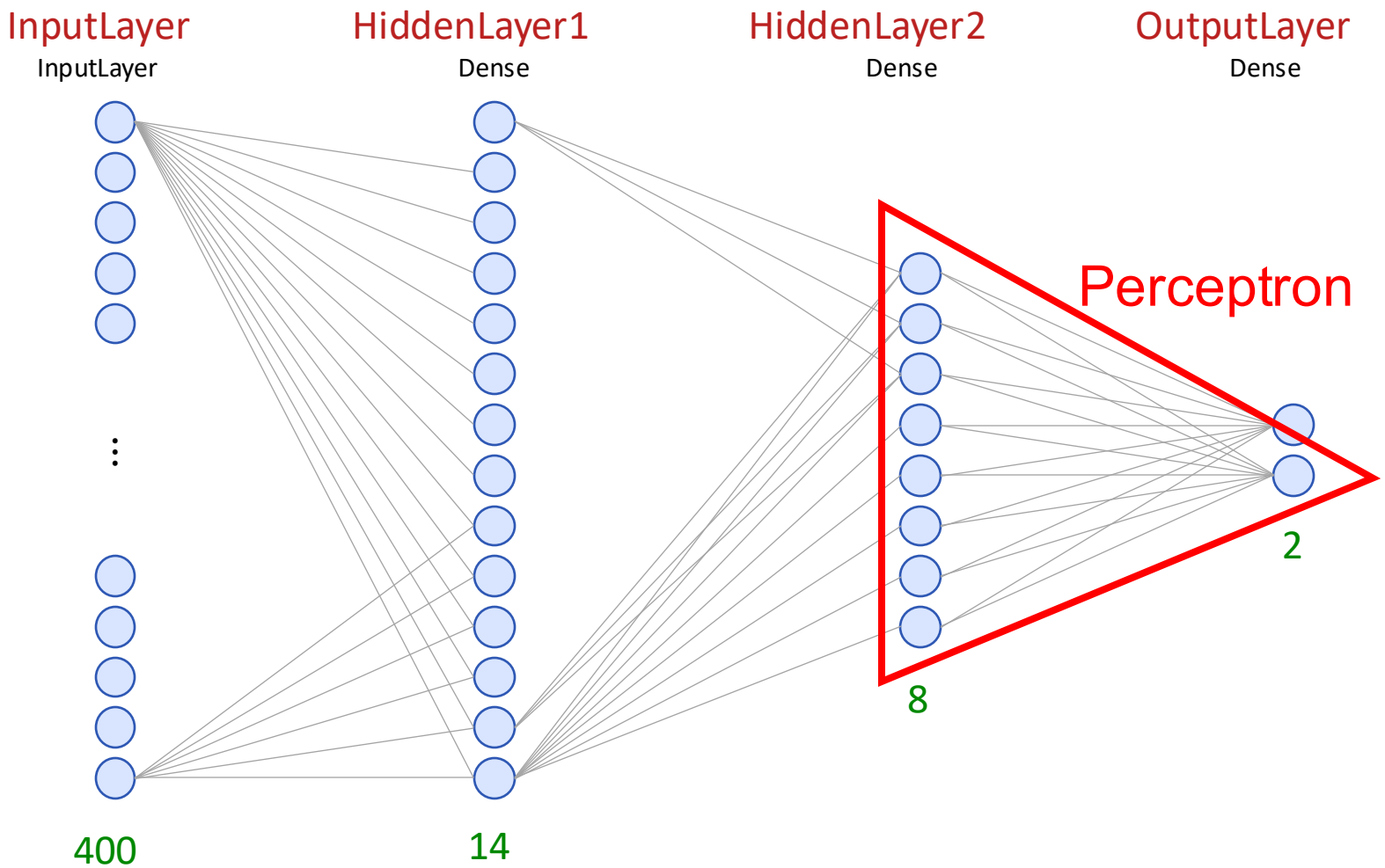
tf.keras.layers.*

- AbstractRNNCell
- Activation
- ActivityRegularization
- Add
- AdditiveAttention
- AlphaDropout
- Attention
- Average
- AveragePooling1D
- AveragePooling2D
- AveragePooling3D
- AvgPool1D
- AvgPool2D
- AvgPool3D
- BatchNormalization
- Bidirectional
- Concatenate
- Conv1D
- Conv1DTranspose
- **Conv2D**
- Conv2DTranspose
- Conv3D
- Conv3DTranspose
- ConvLSTM2D
- Convolution1D
- Convolution1DTranspose
- Convolution2D
- Convolution2DTranspose
- Convolution3D
- Convolution3DTranspose
- Cropping1D
- Cropping2D
- Cropping3D
- Dense
- DenseFeatures
- DepthwiseConv2D
- Dot
- Dropout
- ELU
- Embedding
- Flatten
- GRU
- GRUCell
- GaussianDropout
- GaussianNoise
- GlobalAveragePooling1D
- GlobalAveragePooling2D
- GlobalAveragePooling3D
- GlobalAvgPool1D
- GlobalAvgPool2D
- GlobalAvgPool3D
- GlobalMaxPool1D
- GlobalMaxPool2D
- GlobalMaxPool3D
- GlobalMaxPooling1D
- GlobalMaxPooling2D
- GlobalMaxPooling3D
- InputLayer
- InputSpec
- **LSTM**
- LSTMCell
- Lambda
- Layer
- LayerNormalization
- LeakyReLU
- LocallyConnected1D
- LocallyConnected2D
- Masking
- MaxPool1D
- **MaxPool2D**
- MaxPool3D
- MaxPooling1D
- MaxPooling2D
- MaxPooling3D
- Maximum
- Minimum
- MultiHeadAttention
- Multiply
- PReLU
- Permute
- **RNN**
- **ReLU**
- RepeatVector
- Reshape
- SeparableConv1D
- SeparableConv2D
- SeparableConvolution1D
- SeparableConvolution2D
- SimpleRNN
- SimpleRNNCell
- Softmax
- SpatialDropout1D
- SpatialDropout2D
- SpatialDropout3D
- StackedRNNCells
- Subtract
- ThresholdedReLU
- TimeDistributed
- UpSampling1D
- UpSampling2D
- UpSampling3D
- Wrapper
- ZeroPadding1D
- ZeroPadding2D
- ZeroPadding3D

Neuronal Network Structure

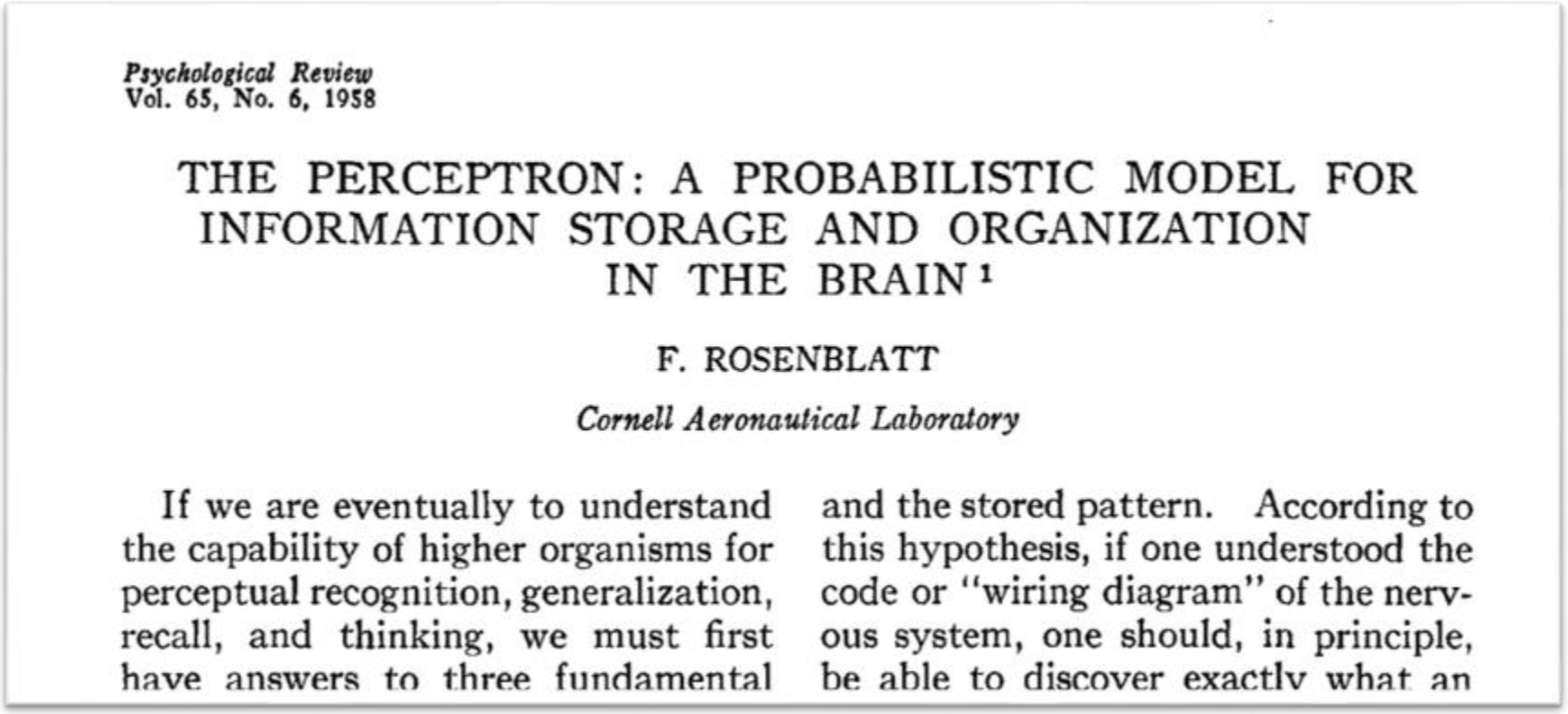


Neuronal Network Structure



Neuronal Networks

What can be trained?



Psychological Review
Vol. 65, No. 6, 1958

THE PERCEPTRON: A PROBABILISTIC MODEL FOR INFORMATION STORAGE AND ORGANIZATION IN THE BRAIN[1]

F. ROSENBLATT

Cornell Aeronautical Laboratory

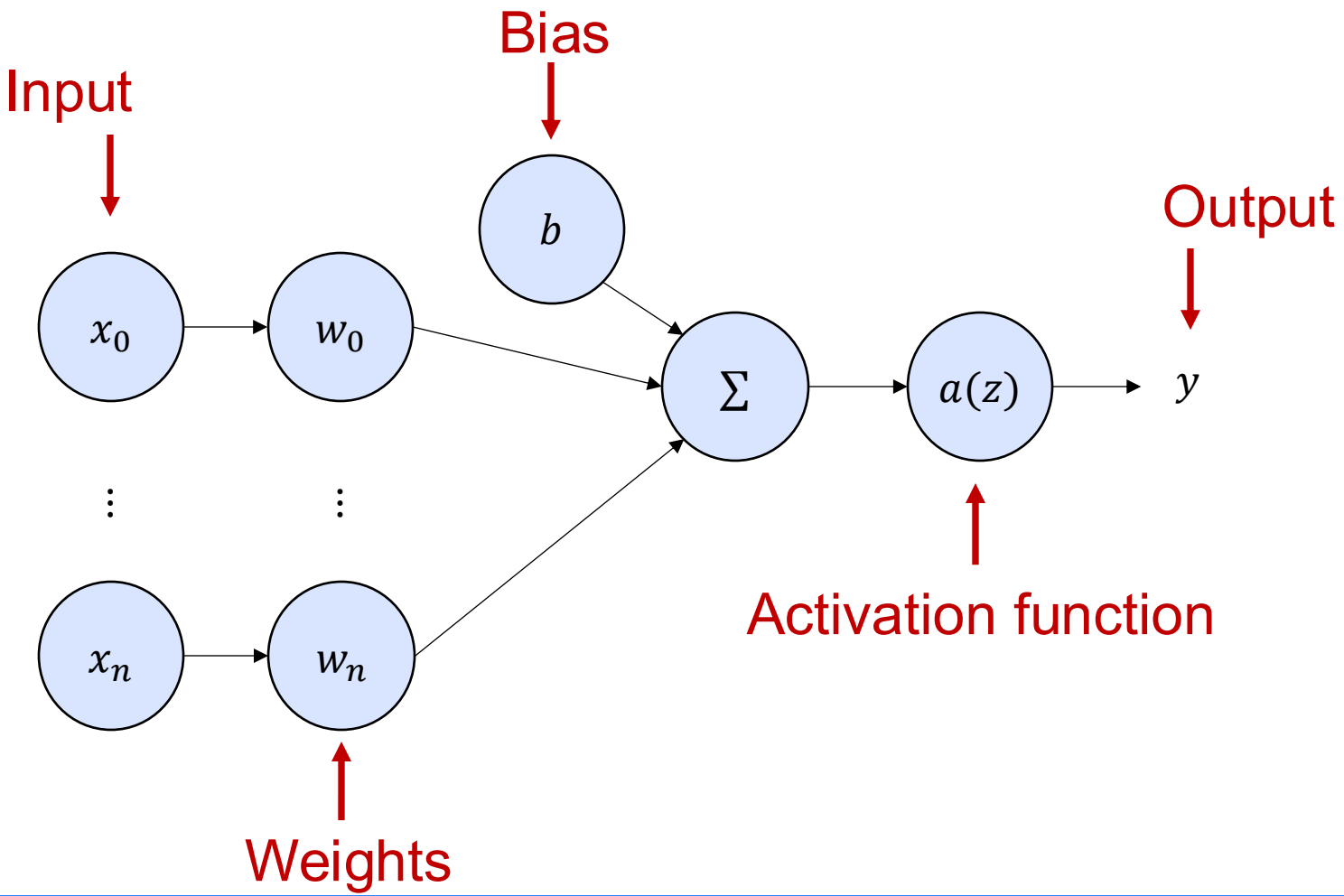
If we are eventually to understand the capability of higher organisms for perceptual recognition, generalization, recall, and thinking, we must first have answers to three fundamental

and the stored pattern. According to this hypothesis, if one understood the code or "wiring diagram" of the nervous system, one should, in principle, be able to discover exactly what an

Frank Rosenblatt. "The perceptron: a probabilistic model for information storage and organization in the brain." *Psychological review* 65, no. 6 (1958): 386. DOI: https://psycnet.apa.org/doi/10.1037/h0042519

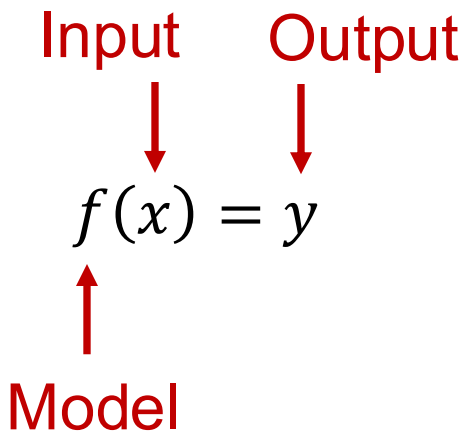
What is a Perceptron?

Single-Layer Perceptron



What can be trained?

Single-Layer Perceptron



What can be trained?

Single-Layer Perceptron

Input

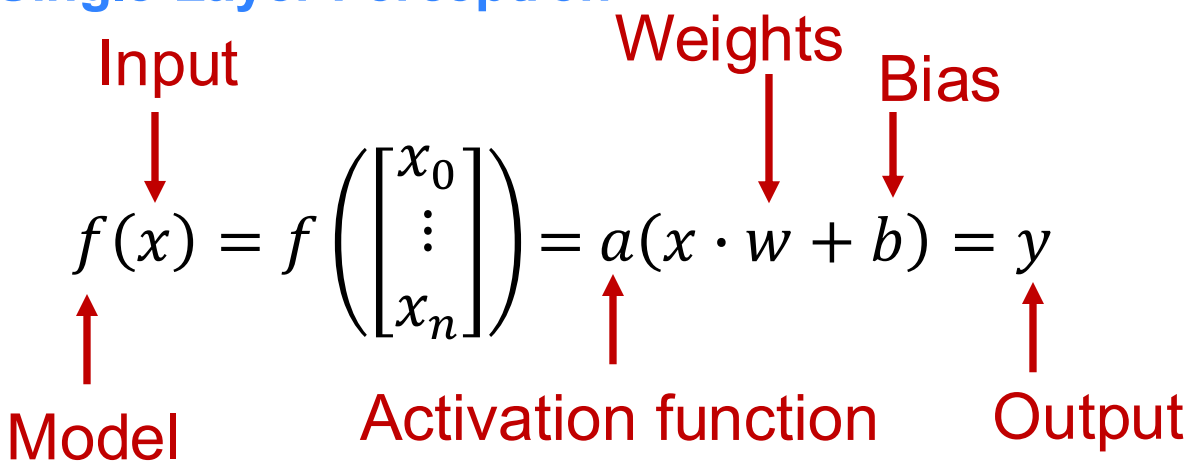
$$f(x) = f\left(\begin{bmatrix} x_0 \\ \vdots \\ x_n \end{bmatrix}\right) = a(x \cdot w + b) = y$$

Model

Output

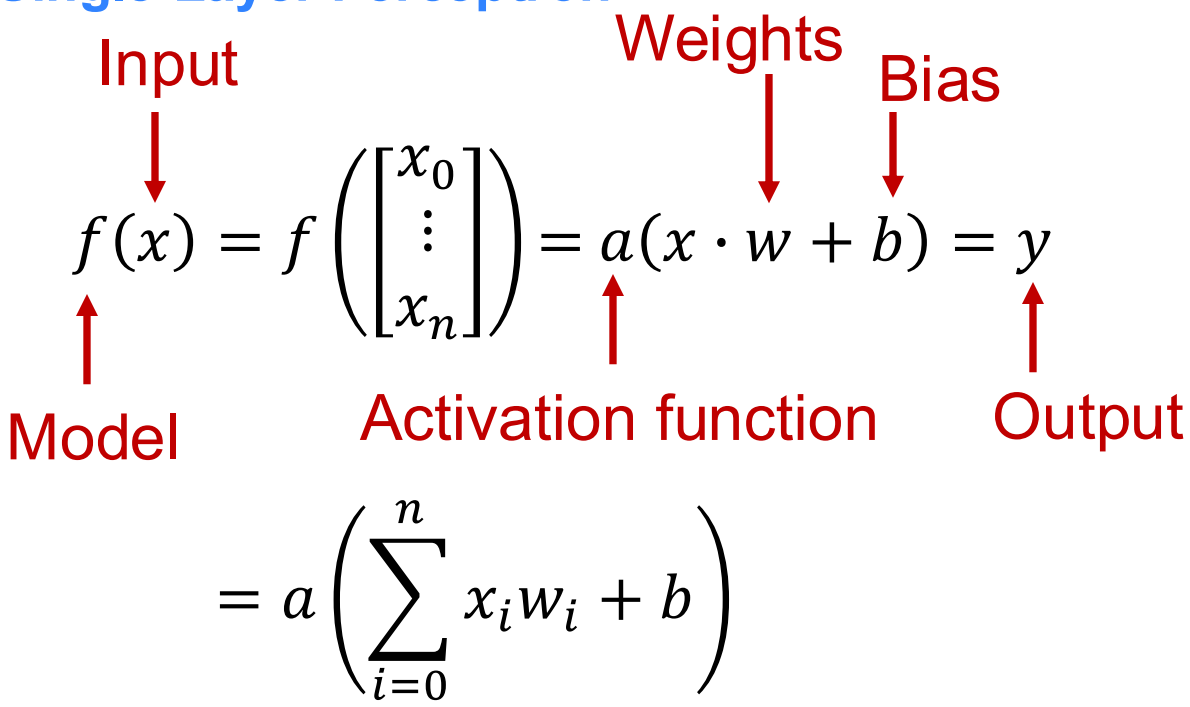
What can be trained?

Single-Layer Perceptron



What can be trained?

Single-Layer Perceptron

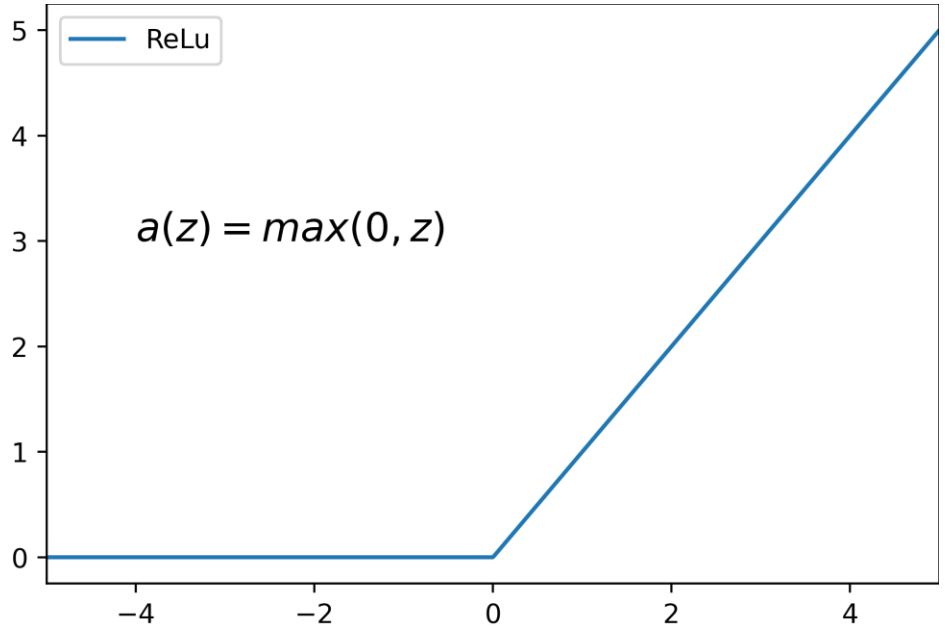


$$= a\left(\sum_{i=0}^{n} x_i w_i + b\right)$$

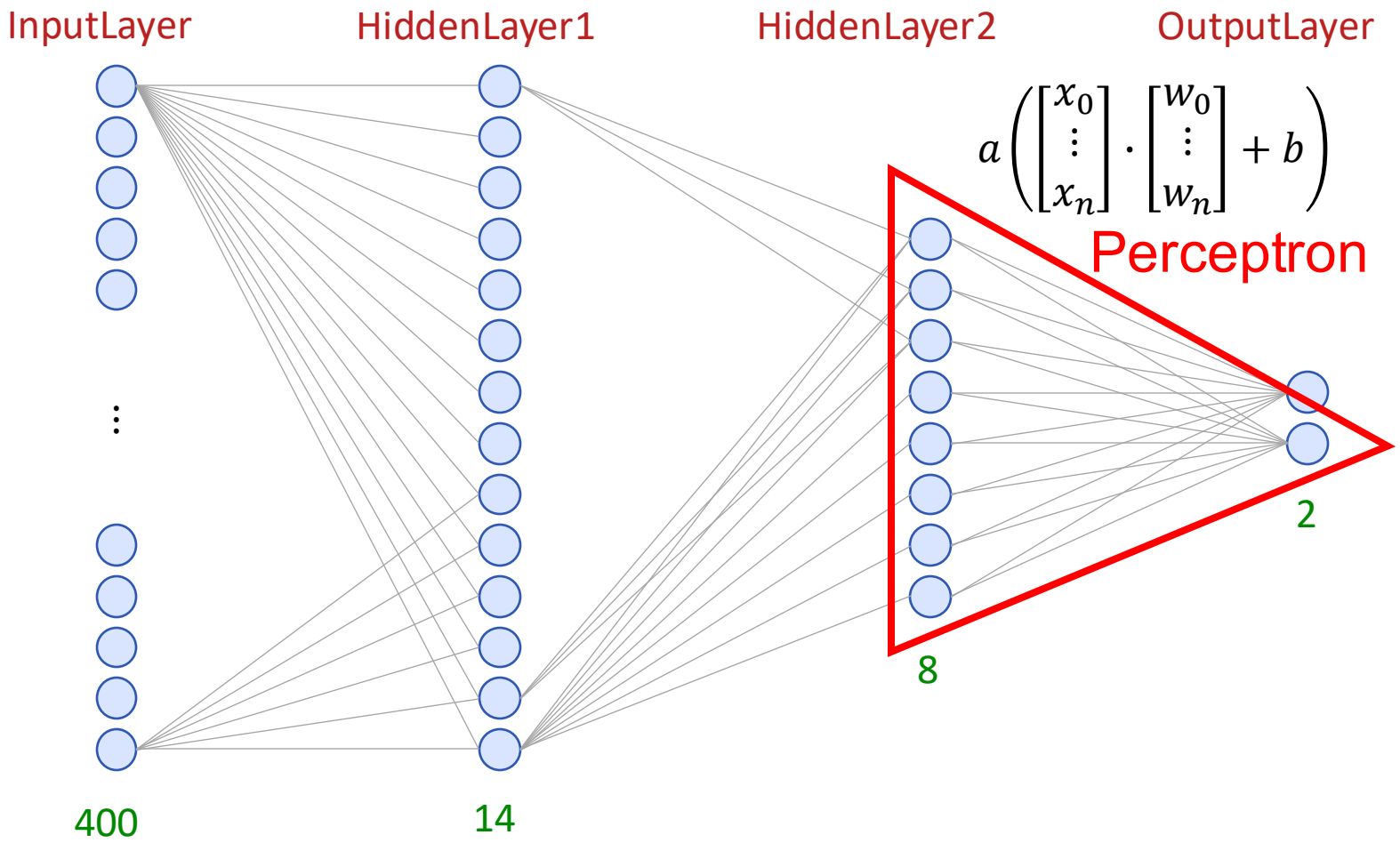
Activation Function

Rectified Linear Unit (ReLU)

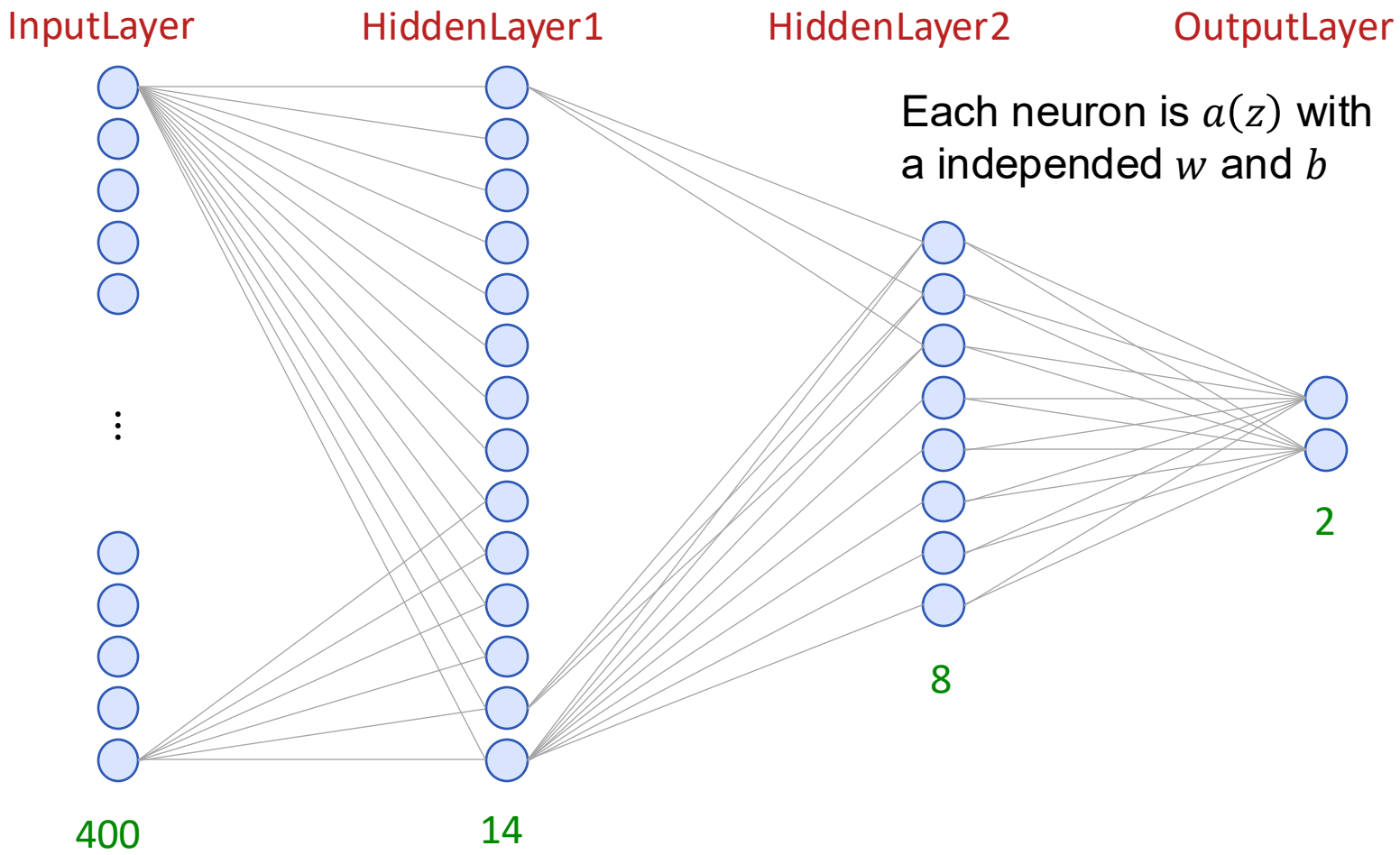
$$a(z) = \max(0, z) = \begin{cases} z & if\ z > 0 \\ 0 & otherwise \end{cases} = \begin{cases} z & if\ x \cdot w + b > 0 \\ 0 & otherwise \end{cases}$$



Neuronal Network Structure

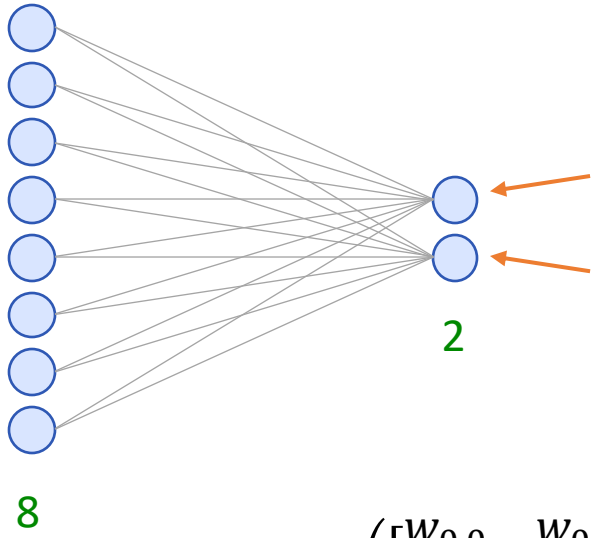


Neuronal Network Structure



Combining Perceptions

Why is it all about fast matrix multiplication?



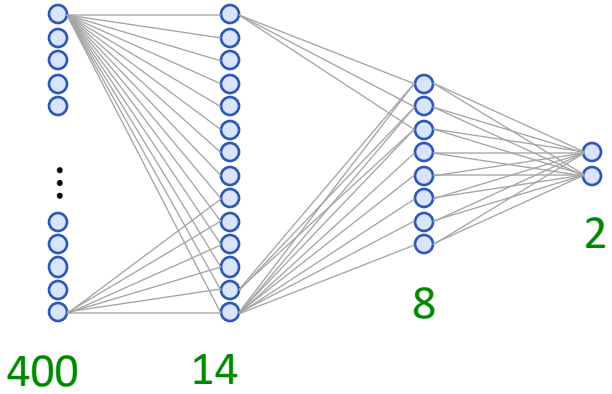
$$a(x \cdot w_0 + b_0) = y_0$$

$$a(x \cdot w_1 + b_1) = y_1$$

$$a\left(\begin{bmatrix} w_{0,0} & w_{0,1} & \dots & w_{0,6} & w_{0,7} \\ w_{1,0} & w_{1,1} & \dots & w_{1,6} & w_{0,7} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \end{bmatrix}\right) = \begin{bmatrix} y_0 \\ y_1 \end{bmatrix}$$

$$a\left(\begin{bmatrix} w_{0,0} & \cdots & w_{0,m} \\ \vdots & \ddots & \vdots \\ w_{n,0} & \cdots & w_{n,m} \end{bmatrix} \begin{bmatrix} x_0 \\ \vdots \\ x_n \end{bmatrix} + \begin{bmatrix} b_0 \\ \vdots \\ b_n \end{bmatrix}\right) = \begin{bmatrix} y_0 \\ \vdots \\ y_n \end{bmatrix}$$

Parameter



- Layers 400, 14, 8, and 2
  - => weights: 400 * 14 + 14 * 8 + 8 * 2 = 5,728
  - => biases: 14 + 8 + 2 = 24
  - => trainable parameter: 5,728 + 24 = 5,752

- Trainable parameters can raise fast
  - Layers 400, 100, 40, 2 => Parameter: 44,222
  - (one model from the walkthough)

Conclusion

Neural Network Structure

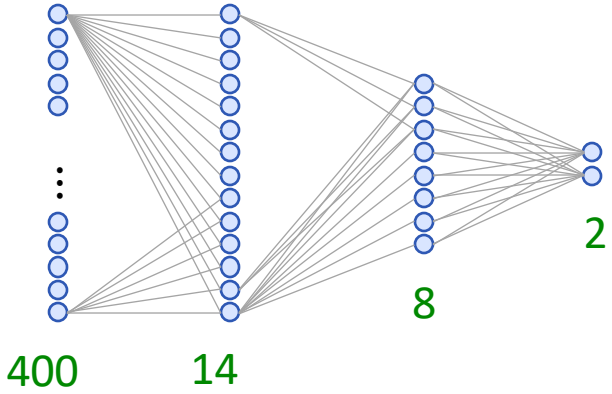
- Perceptron
- Weights
- Biases
- Combining multiple Perceptron
- Trainable Parameter
- Activation Function (e.g. ReLu)

Artificial Intelligence in Interactive Systems

Backpropagation

Sven Mayer

Parameter

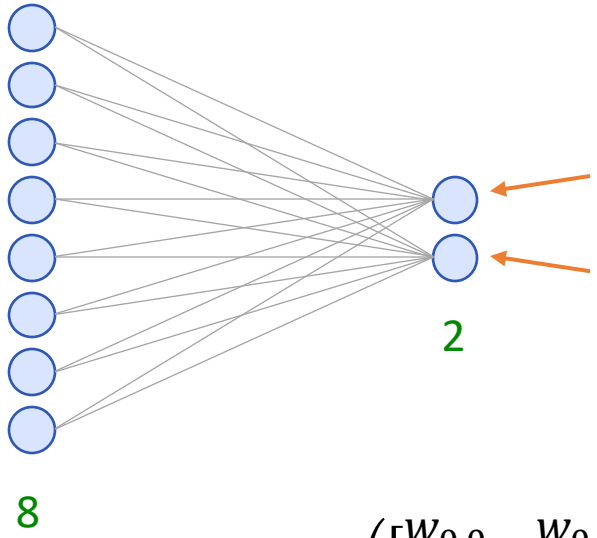


- Layers 400, 14, 8, and 2
  → weights: 400 * 14 + 14 * 8 + 8 * 2 = 5,728
  → biases: 14 + 8 + 2 = 24
  → trainable parameter: 5,728 + 24 = 5,752

- Trainable parameters can raise fast
  - Layers 400, 100, 40, 2 => Parameter: 44,222
  - (one model from the walkthough)

Combining Perceptions

Why is it all about fast matrix multiplication?



$$a(x \cdot w_0 + b_0) = y_0$$

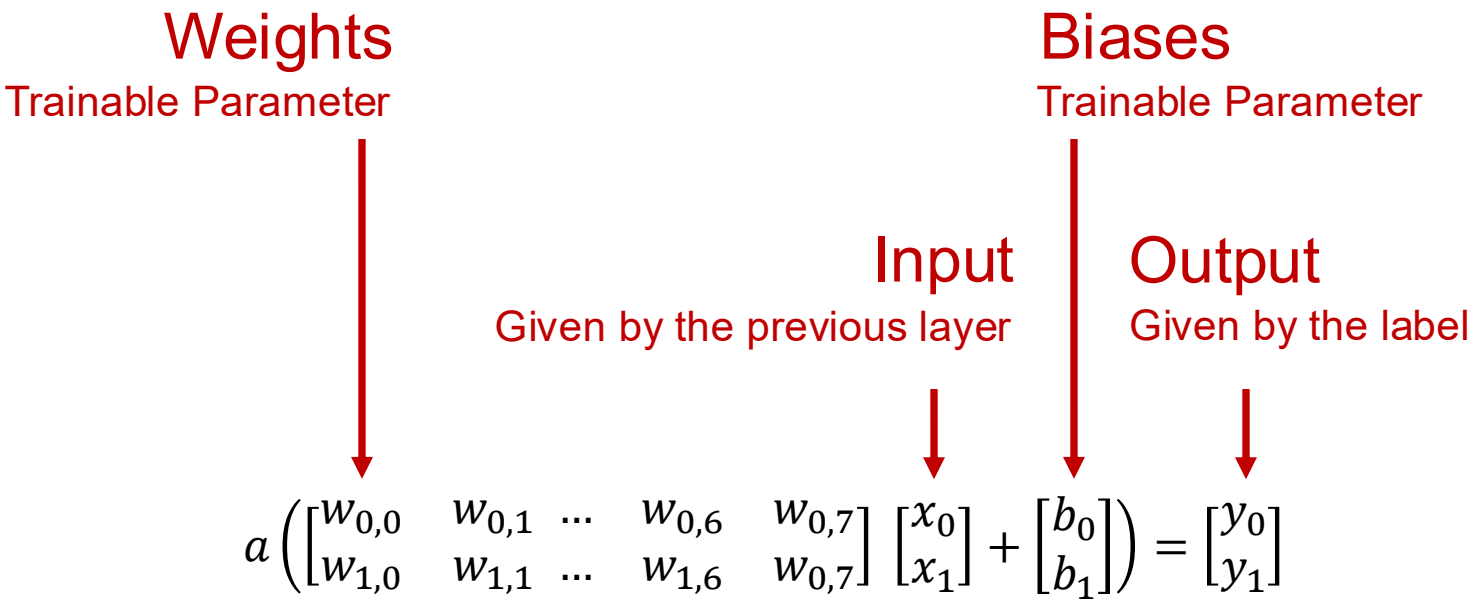
$$a(x \cdot w_1 + b_1) = y_1$$

$$a\left(\begin{bmatrix} w_{0,0} & w_{0,1} & \dots & w_{0,6} & w_{0,7} \\ w_{1,0} & w_{1,1} & \dots & w_{1,6} & w_{0,7} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \end{bmatrix}\right) = \begin{bmatrix} y_0 \\ y_1 \end{bmatrix}$$

$$a\left(\begin{bmatrix} w_{0,0} & \cdots & w_{0,m} \\ \vdots & \ddots & \vdots \\ w_{n,0} & \cdots & w_{n,m} \end{bmatrix} \begin{bmatrix} x_0 \\ \vdots \\ x_n \end{bmatrix} + \begin{bmatrix} b_0 \\ \vdots \\ b_n \end{bmatrix}\right) = \begin{bmatrix} y_0 \\ \vdots \\ y_n \end{bmatrix}$$

Training

Trainable Parameter



$$a\left(\begin{bmatrix} w_{0,0} & \cdots & w_{0,m} \\ \vdots & \ddots & \vdots \\ w_{n,0} & \cdots & w_{n,m} \end{bmatrix} \begin{bmatrix} x_0 \\ \vdots \\ x_n \end{bmatrix} + \begin{bmatrix} b_0 \\ \vdots \\ b_n \end{bmatrix}\right) = \begin{bmatrix} y_0 \\ \vdots \\ y_n \end{bmatrix}$$

Training

1. Initialize w and b randomly
2. Determine how good the model is using a ”cost function”
3. How can we adjust w and b to be better?

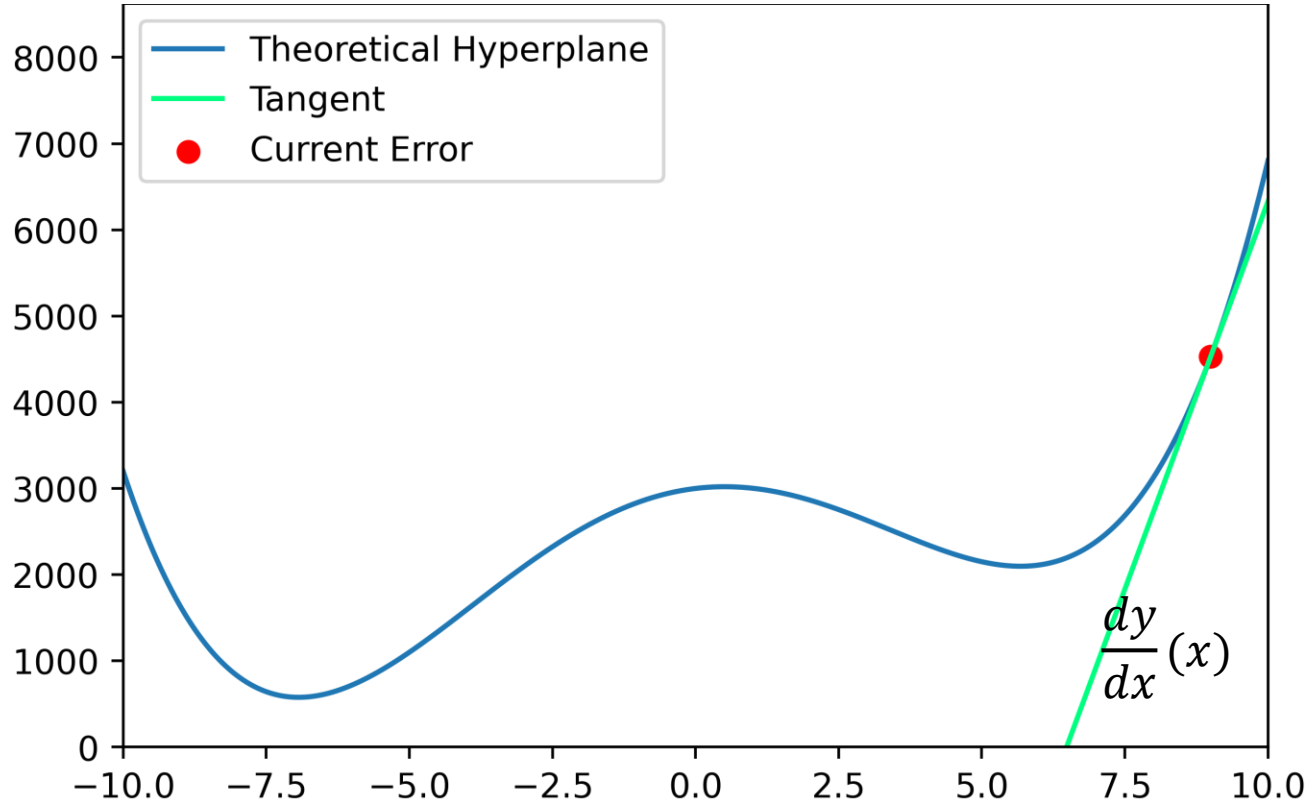
- Cost Functions
  - Squared error
  - RMSE
  - …

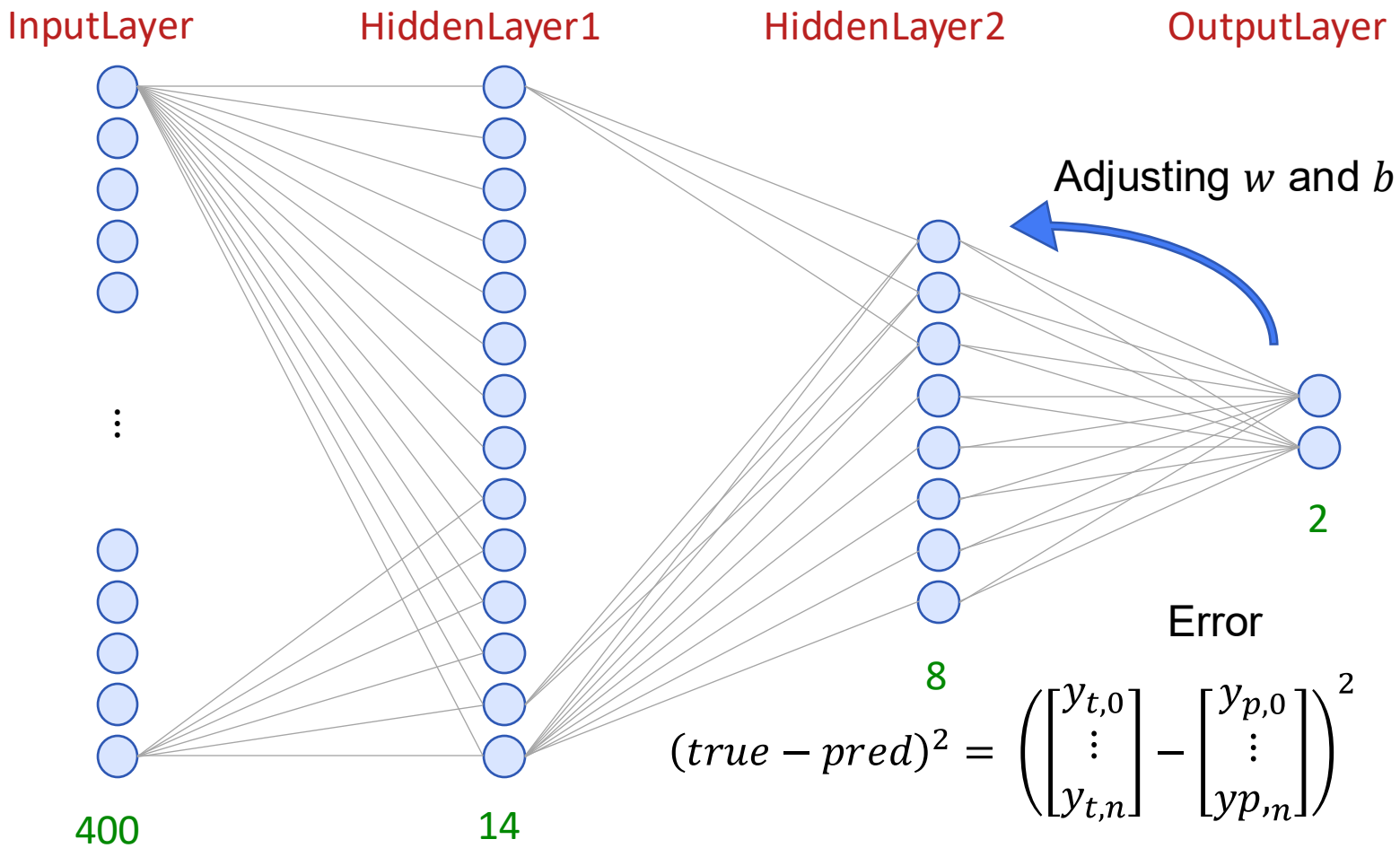
$$\left(\begin{bmatrix} y_{t,0} \\ \vdots \\ y_{t,n} \end{bmatrix} - \begin{bmatrix} y_{p,0} \\ \vdots \\ y_{p,n} \end{bmatrix}\right)^2$$

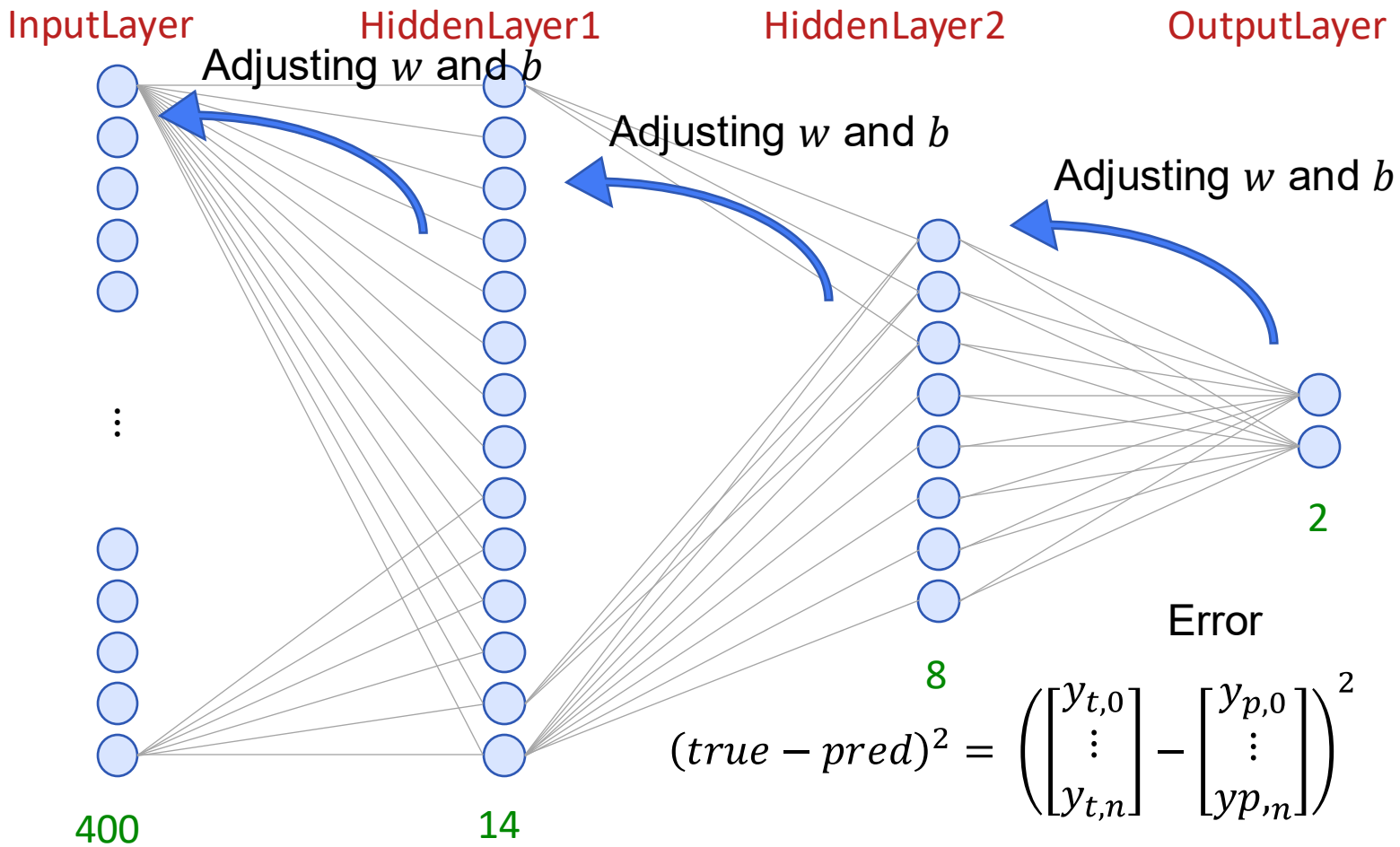
Optimization

“Stochastic Gradient Descent”

- In which direction do we need to go to minimize our cost?




InputLayer
HiddenLayer1
HiddenLayer2
OutputLayer
Adjusting w and b
400
14
8
2
Error
$(true - pred)^2 = \left(\begin{bmatrix} y_{t,0} \\ \vdots \\ y_{t,n} \end{bmatrix} - \begin{bmatrix} y_{p,0} \\ \vdots \\ yp_{,n} \end{bmatrix}\right)^2$


InputLayer
HiddenLayer1
HiddenLayer2
OutputLayer
Adjusting w and b
Adjusting w and b
Adjusting w and b
400
14
8
2
Error
$(true - pred)^2 = \left(\begin{bmatrix} y_{t,0} \\ \vdots \\ y_{t,n} \end{bmatrix} - \begin{bmatrix} y_{p,0} \\ \vdots \\ yp_{,n} \end{bmatrix}\right)^2$

Adjusting w and b

- With the slope, we know how to adjust w and b
- First fast large then small steps

→ “learning rate”

- The gradient can be calculated for each input x
- To not overfit to one x we take more inputs
- Taking all x from takes too long
- In each ”optimization step” we only look at a few x’s

→ ”batch size”

Conclusion

Backpropagation

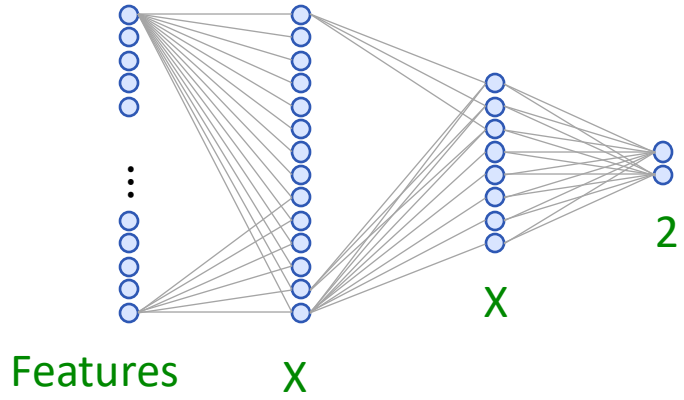
- Backpropagation
- Trainable Parameter: Weights and biases
- Optimizing e.g. Stochastic Gradient Descent
- Loss function e.g. MSE, MAE, RMSE
- Learning rate
- Batch size

Artificial Intelligence in Interactive Systems

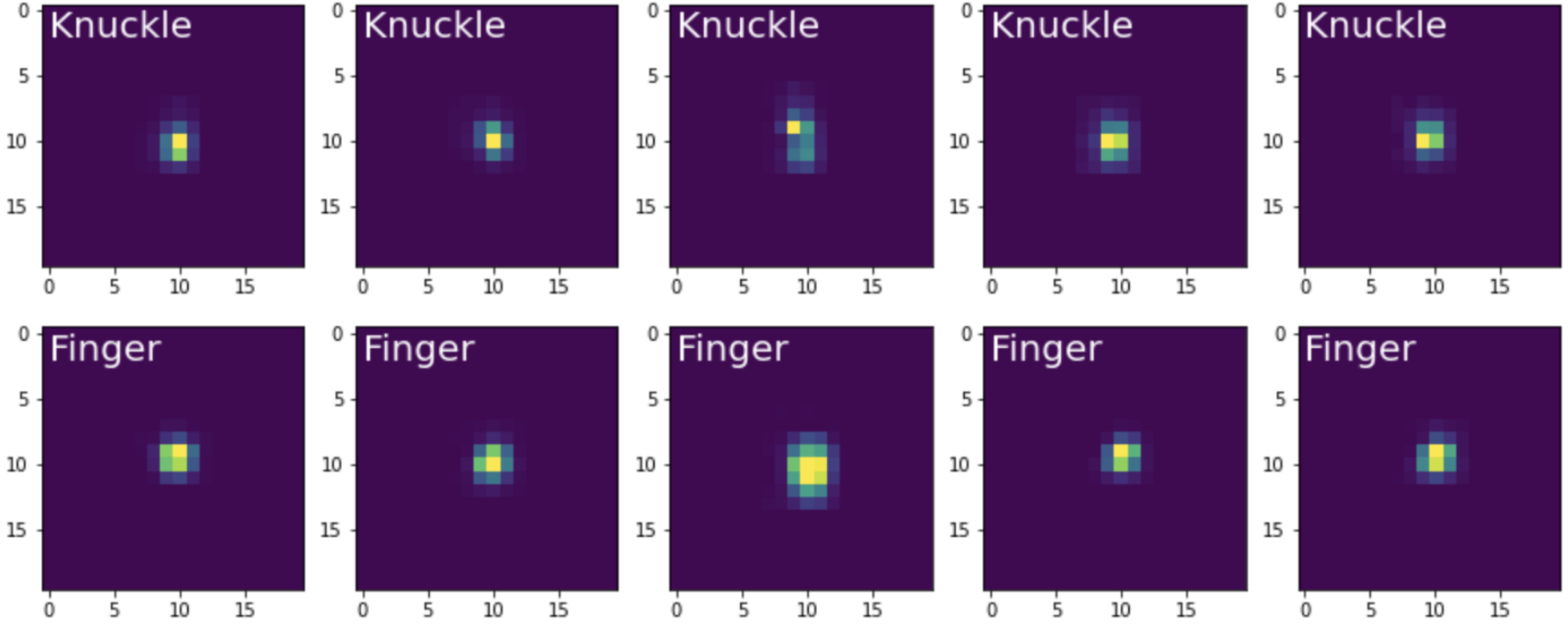
Feature Engineering & Representation Learning

Sven Mayer

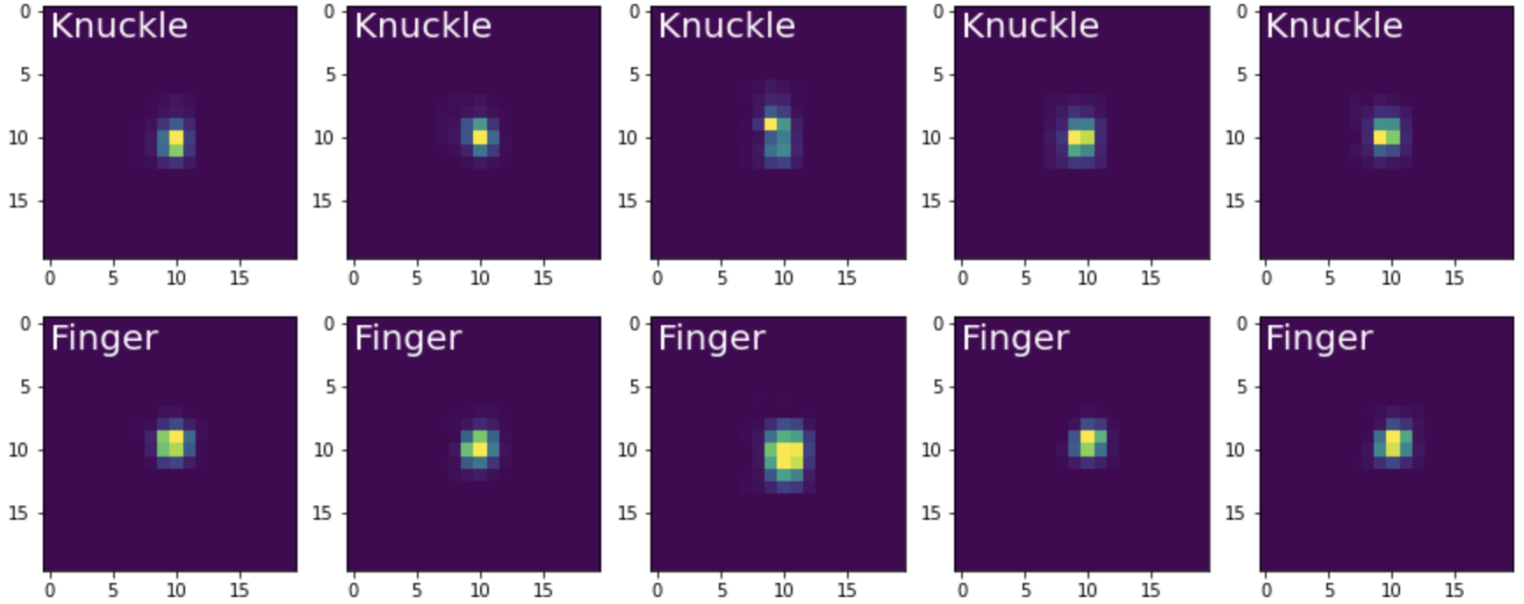
Feature Engineering



- Feature Engineering is often used for non NN ML e.g. SVM
- Requires domain knowledge and “thinking” before training



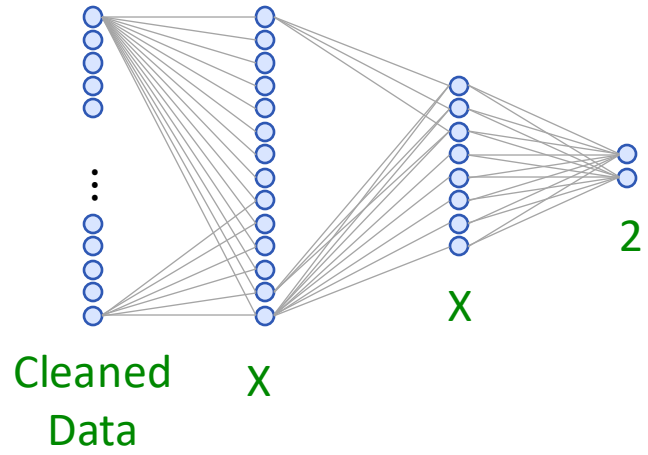
Feature Engineering



- Sum of pixels
- Min/Max value
- Ellipse fitting
  - Radius 1 & 2
  - Theta
- Area of the Convex hull
- …
- … And others we can think of depending on our problem

Representation Learning

- The data gets presented into the model without additional preprocessing
- No domain knowledge
- No thinking
- **The hope** is that the model is doing the “thinking” for you



Pros and Cons

- Feature Engineering
  - Reduced the input data

  → models can be smaller

  → more suitable for “traditional” ML models

- Representation Learning
  - Raw data as input

  → the models need to do more “work”

  → the model has to be larger

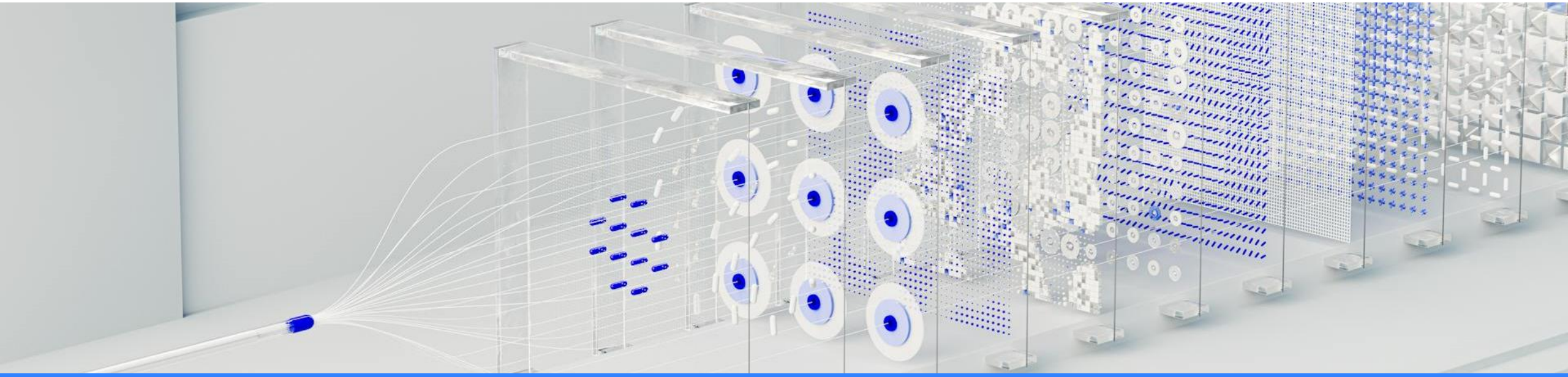
  → harder to train

  → more suitable for NN models

Conclusion

Feature Engineering & Representation Learning

- Feature Engineering
- Representation Learning



Artificial Intelligence in Interactive Systems

Data Preparation

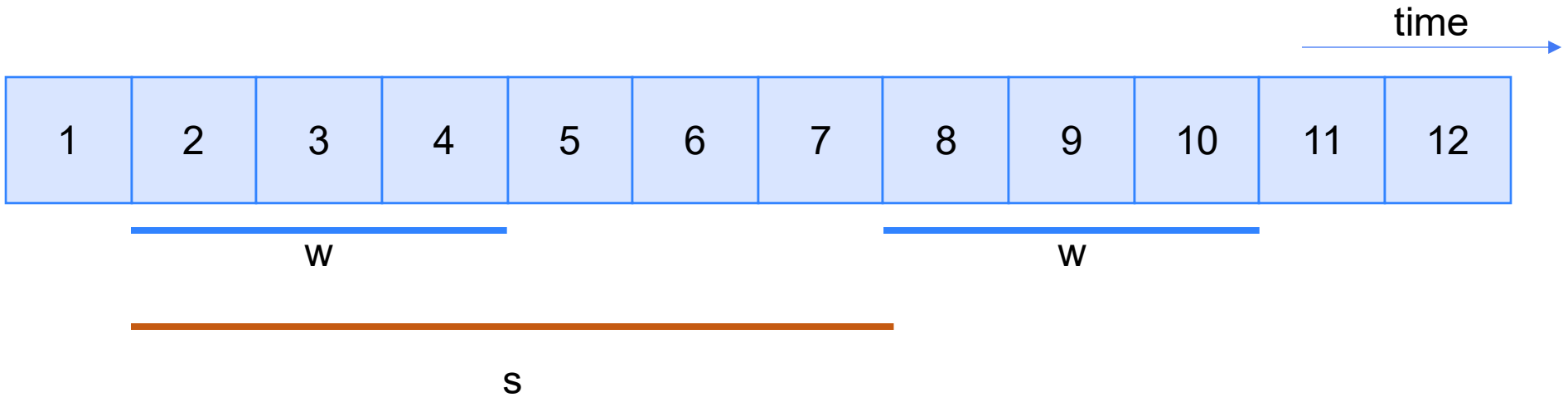
Sven Mayer

Scale Data

- Goal: center mean around 0 or 0.5 to support the activation function
- Common approaches for RGB data (0-255 values):
  - value / 255 => scaled to 0 to 1
  - value / 127.5 -1 => scaled to -1 to 1
- Advanced Normalization
  - z-score normalization [1]
  - Interquartile range (IQR) scaler [2]

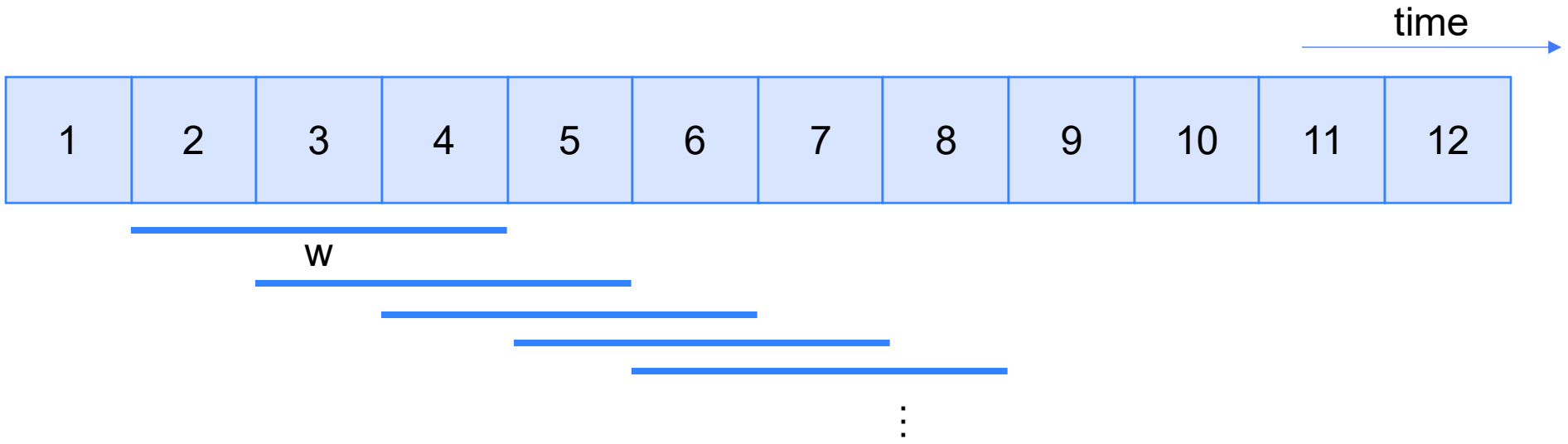
[1] https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html
[2] https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.RobustScaler.html

Sliding Window Technique

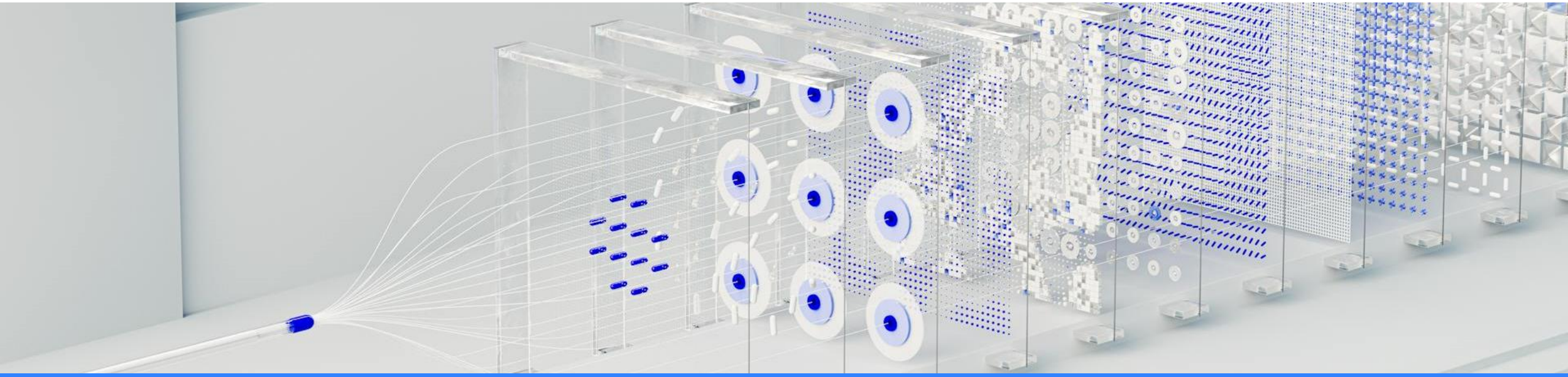


- w = window length
- s = stride aka “step size”

Sliding Window Technique



- w = window length
- s = stride aka “step size”



Practical Machine Learning

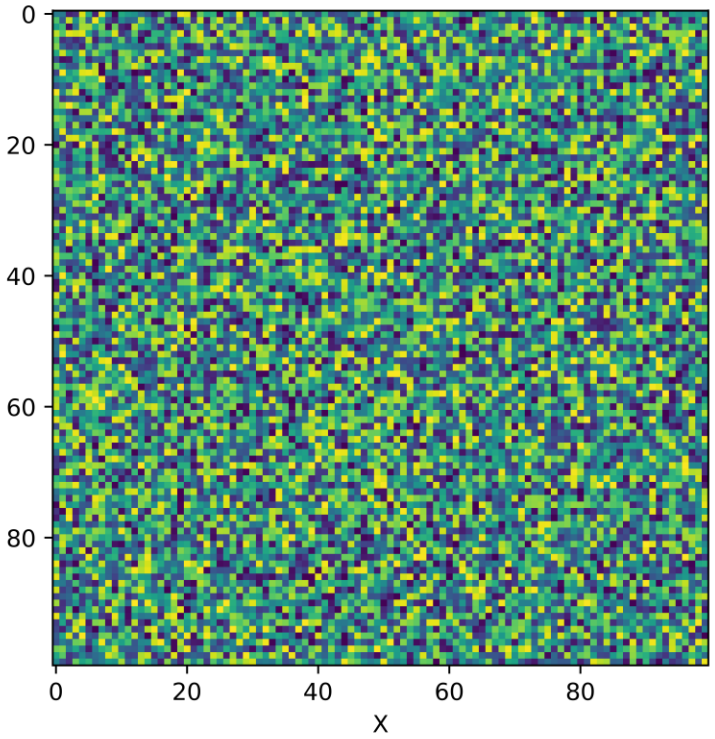
Data Argumentation

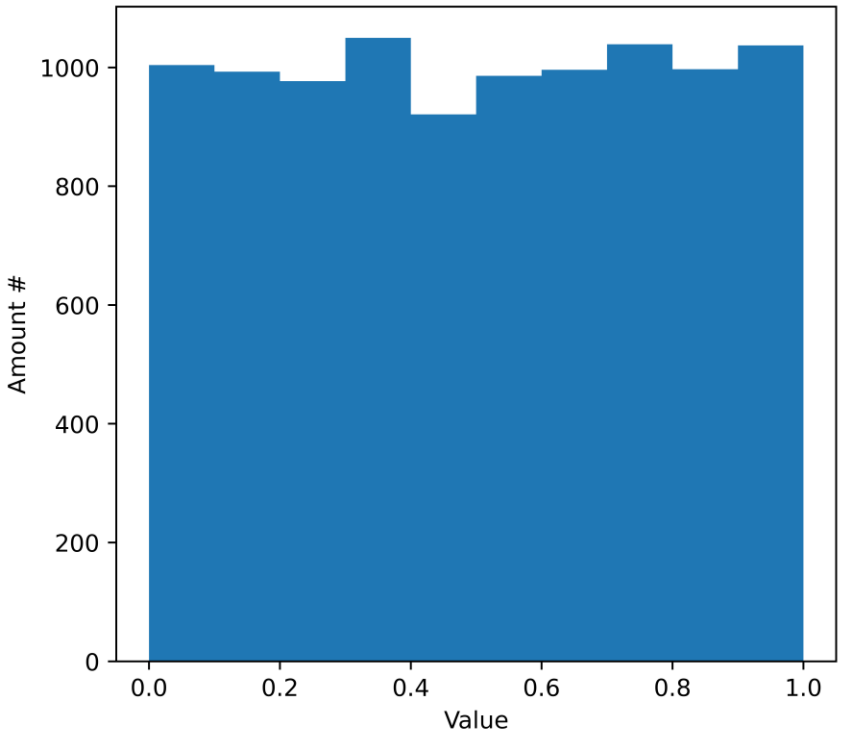
Sven Mayer

Not enough data is a constant issue in machine learning,
Data Argumentation tries to address this issue.

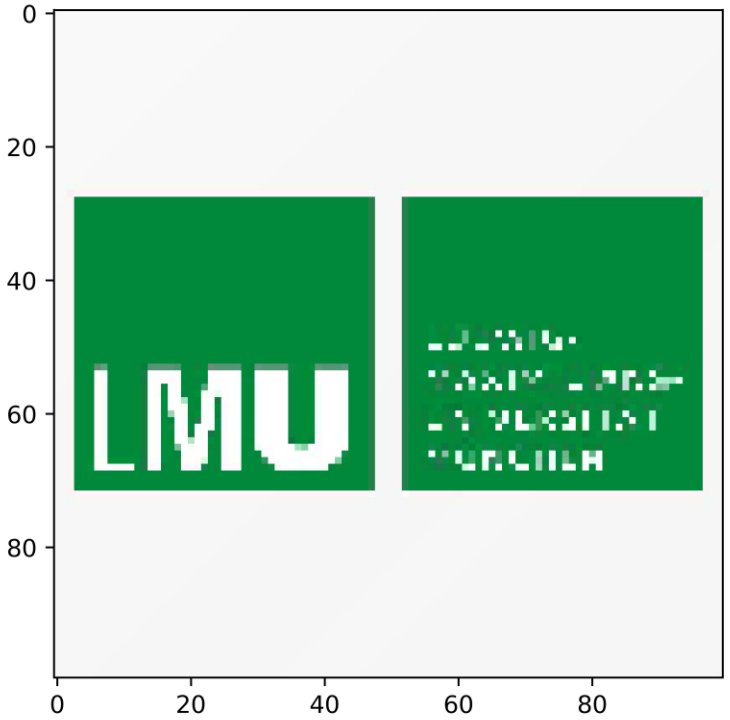
Adding Noise to the Data

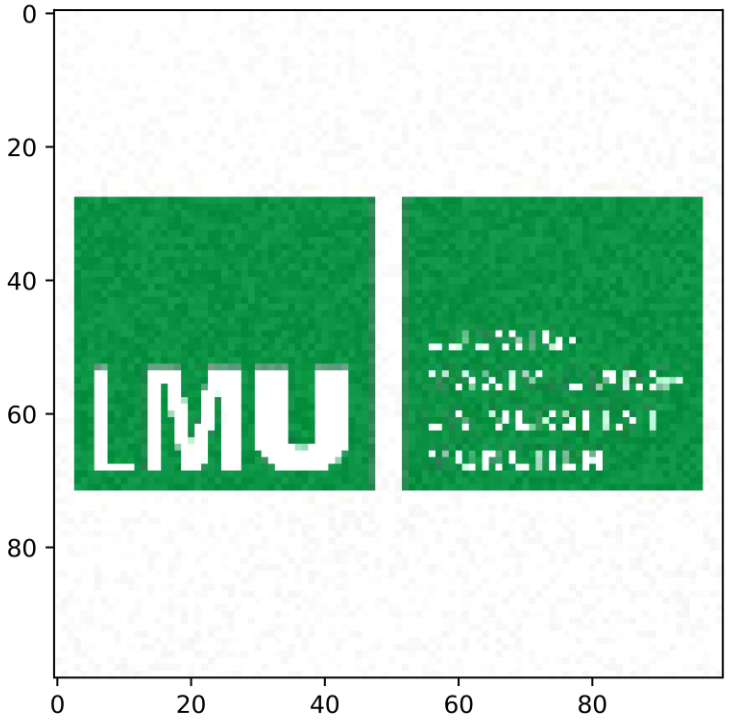
”true” random noise





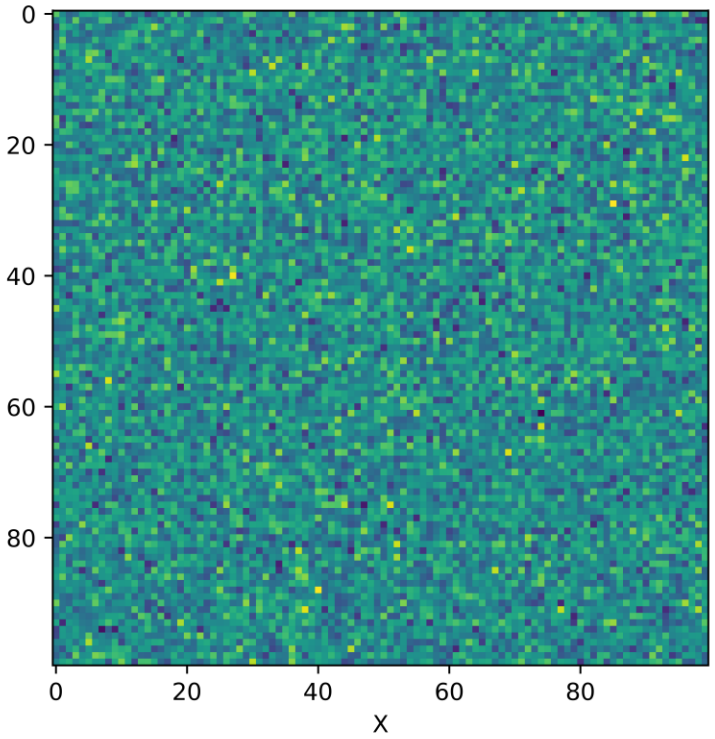
Adding Noise to the Data

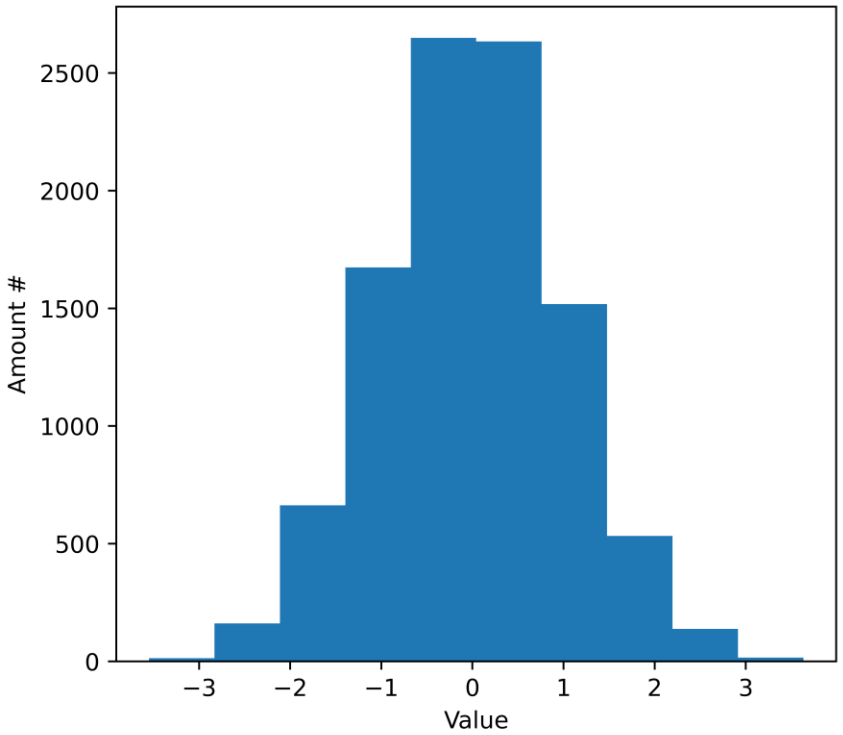




Adding Noise to the Data

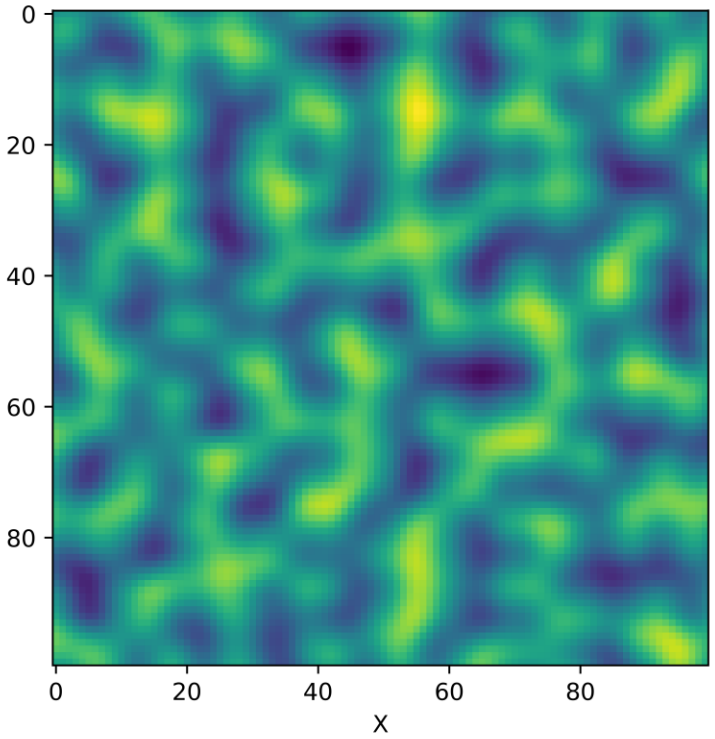
Normal random noise

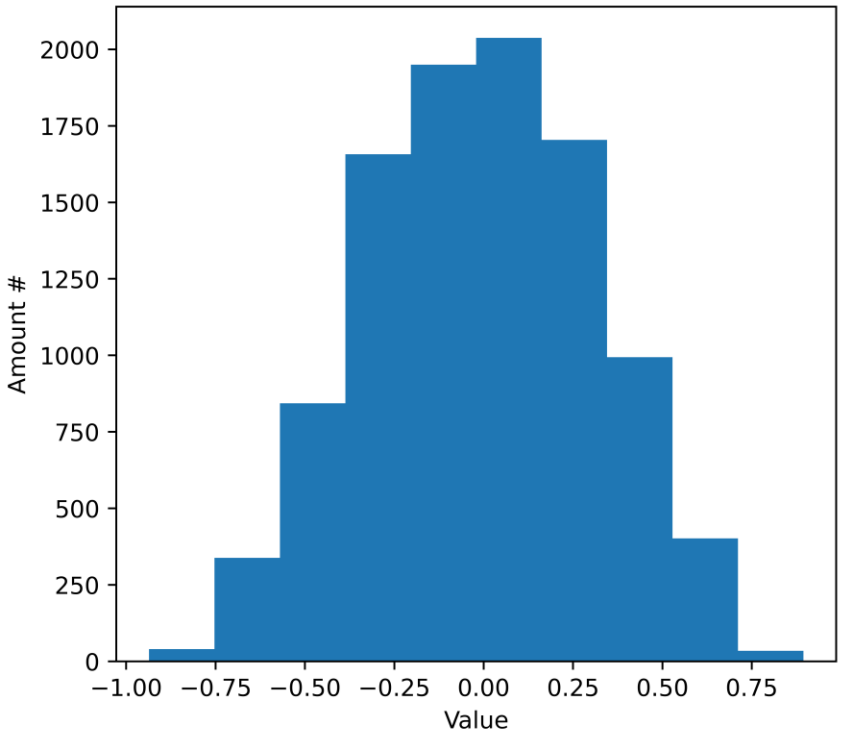




Adding Noise to the Data

Perlin noise





Data Argumentation Ideas

For Images

- Translate
- Zoom/Crop
- Rotate
- Flip

- Filling methods for “unknown” information is key.
  - Fill with “zero”.
  - Always zoom in.
  - Fill with other image information.